

Practical 3 : CA)

* Problem Statement :

→ WAP to Print array elements in reverse order using Dynamic memory Allocation

* Algorithm :

1. Start.
2. Input the number of elements n .
3. Allocate memory dynamically for n integers using `malloc()`.
4. Check if memory allocation is successful
- if not successful, display "memory Allocation failed" and Stop.
5. Read n array elements from the user.
6. Set the Pointer to the first element of the dynamically allocated array.
7. Traverse the array from index $n-1$ to 0 .
8. Print each element in reverse order using Pointer arithmetic.
9. Free the allocated memory using `free()`.
10. Stop.

* code:

```
#include <iostream>
using namespace std;
```

```
int main() {
    int n;
```

```
    cout << "Enter number of elements:";
    cin >> n;
```

```
    int *arr = (int*) malloc (n * sizeof(int));
```

```
    if (arr == NULL) {
```

```
        cout << "Memory Allocation
        Failed" << endl;
        return 0;
```

```
    cout << "Enter" << n << "elements:";
    for (int i = 0; i < n; i++) {
```

```
        cin >> *arr + i;
```

```
    cout << "Array in reverse order:";
    for (int i = n - 1; i >= 0; i--)
```

```
    {
```

```
        cout << (*arr + i) << " ";
```

```
    }
```

```
    free(arr);
```

```
    return 0;
```

```
}
```


187 : 157 20071700009

=> Output :

Enter number of elements : 3

Enter 3 elements :

3 2 1

Array in reverse order :

1 2 3

Practicle (2): (C)

* Problem Statement:

-> Demonstrate the concept of call by value.

* Algorithm:

1. Start
2. Declare an integer variable u .
3. Initialize u with a value.
4. Display the value of u .
5. Call the function CallByValue(u).
6. Inside the function:
 - Receive a copy of u .
 - Modify the copied value.
 - Display the modified value.
7. Return to the main() function.
8. Display the original value of u .
9. Observe that the original value remains unchanged.
10. Stop.

* code : 159099142. mail id or *

```
#include <iostream>
using namespace std;
```

```
void callByValue (int x) {  
    x = x + 10;  
}
```

3

~~cout << "value"~~
 cout << "value inside function: " << x << endl;

the minor of k is 19210 is

2. $\text{int } a = 5;$

cout << "value before i = function"

2 30 2290597 cell: "u<<u9<<end;"

сен Виче сави 1500 -

core < "value" after function

call: " <<u <<ener;

1.3301 return 50; // 1000000

2020-2021

5912150m 1992 2001

$\Rightarrow$ Over Profit :

value before function call: 5

value inside function : 15
value after function call

value after function call: 5

* Problem Statement: *

→ Demonstrate the concept of call by reference

* Algorithm:

01 + x = x

1. Start

2. Declare an integer variable x .

3. Initialize x with a value.

4. Display the value of x .

5. Call the function callByReference(x) passing the address of x .

6. Inside the function:

- Receive the address of x using a Pointer

- modify the value stored at that address.

- Display modified value.

7. Return to the main() function.

8. Display the updated value of x .

9. observe that the original variable has been modified.

10. Stop.

=> Code:

```
#include <iostream>
using namespace std;

void callByReference (int *x) {
    *x = *x + 10;
    cout << "value inside function: " << *x << endl;
}

int main () {
    int b = 5;

    cout << "value before function call: " << b << endl;

    callByReference (&b);

    cout << "value after function call: " << b << endl;
    return 0;
}
```

=> Output

```
value before function call: 5
value inside function : 15
value after function call: 15
```